Plan Maestro de Implementación para Claude: Captura y Renderizado Dual RealSense

> **Propósito del Documento:**
> Este documento consolida todo el contexto técnico, hallazgos de hardware, decisiones de arquitectura y código base desarrollado en los planes 01 a 05 de `documentacion/planes/`. Está redactado como una **guía técnica exhaustiva y ejecutable** para que un modelo de IA avanzado (Claude) o un ingeniero pueda implementar de inicio a fin la captura y renderizado dual sin ambigüedades ni dependencias perdidas.

---

1. Contexto del Proyecto y Estado Actual del Repositorio

A. Repositorio Unificado

- **Punto de Entrada Único:** `app.py` ejecuta `live3d.server.main()`, sirviendo la aplicación web interactiva en `http://localhost:5002`.
- **Servidor Asíncrono:** Desarrollado con `aiohttp`, gestiona tanto el WebSocket binario de geometría 3D (`/ws`) como la API REST (`/api/*`).
- **Visor 3D WebGL:** Basado en Three.js con Vertex Shaders en GPU (`live3d/web/js/shaders.js`, `viewer.js`, `stream.js`, `main.js`). Reconstruye nubes de puntos y mallas directamente en la tarjeta gráfica del cliente sin saturar la CPU.
- **Procesamiento Geométrico 3D:** Utiliza Open3D para alineación ICP, registro multiescaneo TSDF, reconstrucción Poisson y cálculo de nubes fusionadas.

B. Hardware Validado y Conectado en Vivo

El sistema cuenta actualmente con dos cámaras físicas conectadas y operativas por USB 3.2: 1. **Cámara 1 (Master):** Intel RealSense D455 · Serial `207322250777` · Firmware `5.17.3.10`. 2. **Cámara 2 (Slave):** Intel RealSense D455 · Serial `203522251709` · Firmware `5.12.7.150`. * **Rendimiento Comprobado:** Ambas cámaras transmiten concurrentemente a **30.0 FPS y 30.2 FPS** a resolución 848x480 (profundidad Z16 y color RGB8) sin pérdida de fotogramas.

---

2. Restricciones Técnicas Críticas y "Gotchas" de Hardware

Cualquier implementación sobre este sistema debe respetar estrictamente estos cuatro principios descubiertos durante la fase de validación empírica:

1. **Regla de Sincronización (`inter_cam_sync_mode = 0`):**

2. `inter_cam_sync_mode` **NO se puede cambiar mientras el pipeline está activo** (`RuntimeError: Cannot change Inter-camera HW synchronization mode while streaming!`).
3. El modo `2` (Slave por hardware) solo funciona si hay un cable físico de sincronización conectado a los pines GPIO (Pines 1 y 2 a 1.8V TTL). Si no hay cable, el sensor se congela esperando un pulso externo.
4. **Mandato:** Operar siempre con `sync_mode = 0` (software sync) usando los timestamps de hardware de librealsense (`frame.get_timestamp()`).
5. **Mitigación Óptica de Láser IR (Crucial para Calidad de Profundidad):**
6. Si ambas cámaras proyectan sus patrones de puntos infrarrojos a la vez, los proyectores interfieren entre sí generando ruido en la correspondencia estéreo.
7. **Mandato:** Cámara 1 (Master) debe tener `emitter_enabled = 1`. Cámara 2 (Slave) debe tener `emitter_enabled = 0`. La Cámara 2 aprovecha la textura proyectada por la Cámara 1.
8. **Reseteo de Endpoints USB en Windows:**
9. Si un proceso en Windows termina de forma abrupta mientras un pipeline de librealsense está abierto, el controlador USB queda bloqueado (`RuntimeError: Frame didn't arrive within 5000ms`).
10. **Mandato:** Siempre invocar `dev.hardware_reset()` y pausar 5-6 segundos antes de reiniciar el servicio si hubo una salida forzada.
11. **Multiprocesamiento y Desacoplamiento de Hilos:**
12. Cada cámara debe tener su propio `threading.Thread` dedicado ejecutando su propio `wait_for_frames()`. El acceso a los datos de cuadros debe sincronizarse mediante `threading.Lock`.

---

3. Plan de Implementación Paso a Paso

FASE 1: Multiplexación de Red para Streaming Dual Simultáneo

Objetivo:

Emitir los cuadros de ambas cámaras por el mismo WebSocket (`/ws`) de forma continua para que el cliente WebGL reciba los dos flujos sin abrir dos conexiones separadas.

Archivos a Modificar:

1. `live3d/protocol.py`:

2. En la cabecera `RSF1` (32 bytes), utilizar el campo `flags` (offset 28..31) o `meta_id` (offset 16..17) para codificar el índice de la cámara:
    - `flags = 0` $\implies$ Cámara 1.
    - `flags = 1` $\implies$ Cámara 2.

3. `live3d/capture.py` (`DualRealSenseSource`):
4. Actualmente, el worker de la cámara inactiva actualiza `self._cam_data` pero no emite.
5. Modificar para que en modo dual, **ambos workers** invoquen `self._emit_dual(cam_idx, depth_np, color_np, cam_meta, s, t0)`.
6. El método `_emit_dual` empaca cada cuadro con su flag de cámara correspondiente.
7. `live3d/server.py` (`Hub.publish`):
8. Distribuir los paquetes de ambas cámaras a los clientes suscritos.
9. Mantener el control de flujo individual para cada cámara para evitar que un retraso en una ralentice la otra.

FASE 2: Nodos de Renderizado Three.js en el Visor WebGL

Objetivo:

Renderizar las dos fuentes de geometría en el mismo espacio 3D de Three.js.

Archivos a Modificar:

1. `live3d/web/js/viewer.js`:

2. Crear la clase `CameraRenderNode`:
   - Contiene una malla de puntos (`THREE.Points`) y una malla de polígonos (`THREE.Mesh`).
   - Tiene su propio `ShaderMaterial` clonado con sus uniformes específicos:
   - `uDepth`, `uColor`, `uDIntr`, `uCIntr`, `uDScale`, `uRange`, `uCamIdx`.
   - Permite aplicar una transformación espacial rígida 4×4 mediante `group.matrix.fromArray(...)`.

3. Instanciar en `Viewer`:
   - `this.cam1Node = new CameraRenderNode(this.scene, 0, true);` (en el origen $[0,0,0]$).
   - `this.cam2Node = new CameraRenderNode(this.scene, 1, false);` (transformado por $T_{2 \to 1}$).

4. `live3d/web/js/shaders.js`:

5. Añadir soporte para el uniforme `uCamIdx` y modos de colorimetría:
   - Si `uColorMode == 4` (Diagnóstico de Solape):
   - Si `uCamIdx == 0`: Salida color Cian translúcido `vec4(0.1, 0.85, 0.95, 0.8)`.
   - Si `uCamIdx == 1`: Salida color Magenta/Ámbar `vec4(0.95, 0.25, 0.75, 0.8)`.

6. Conversión de Coordenadas RealSense a Three.js:

7. Dado $T_{2 \to 1}^{\text{RS}}$ de `config/dual_camera_config.json`: $$C = \begin{bmatrix} 1 & 0 & 0 & 0 \ 0 & -1 & 0 & 0 \ 0 & 0 & -1 & 0 \ 0 & 0 & 0 & 1 \end{bmatrix}$$ $$T_{\text{ThreeJS}} = C \cdot T_{2 \to 1}^{\text{RS}} \cdot C^{-1}$$
8. Inyectar T_{ThreeJS} en `cam2Node.group.matrix`.

FASE 3: Despachador de Flujos en el Frontend (`stream.js` y `main.js`)

Objetivo:

Decodificar los paquetes binarios entrantes y asignar cada imagen al nodo de cámara correcto.

Archivos a Modificar:

1. `live3d/web/js/stream.js`:
2. Leer `flags` (offset 28) o `meta_id` del paquete binario.
3. Extraer `camIdx = (flags & 1)`.
4. Notificar al callback de cuadro: `onFrame(depthBuffer, colorBitmap, camIdx)`.
5. `live3d/web/js/main.js`:
6. Conectar `onFrame` con `viewer.setFrame(depthBuffer, colorBitmap, camIdx)`.
7. Manejar la llegada de metadatos independientes (`meta_cam1` y `meta_cam2`).

FASE 4: Controles de Interfaz y Modos de Visualización

Archivos a Modificar:

1. `live3d/web/index.html`:
2. En el contenedor `#dual-cam-controls`, sustituir el selector simple por un menú de modos:
   - **Dual Fusionado (Ambas en 3D)** [Predeterminado].
   - **Split-Screen (Pantalla Dividida 50/50)**.
   - **Solo Cam 1**.
   - **Solo Cam 2**.
3. Añadir selector de colorimetría de diagnóstico: "Color Real / Diagnóstico de Solape (Cian-Magenta)".
4. Añadir panel desplegable **"Ajuste Fino de Calibración"**:
   - Sliders de Traslación $\Delta X, \Delta Y, \Delta Z$ ($\pm 10\ \text{cm}$).
   - Sliders de Rotación Yaw, Pitch, Roll ($\pm 15^\circ$).
   - Botón "Guardar Alineación" que invoca `POST /api/dual/extrinsics`.

5. `live3d/web/js/main.js`:
6. Event listeners para los sliders que actualizan en tiempo real la matriz de `cam2Node` en la GPU sin reiniciar el servidor.

FASE 5: Integración con el Addon de Blender (`RealSense_Importer.py`)

Archivos a Modificar:

1. `RealSense_Importer.py`:
2. Actualizar el operador `REALSENSE_OT_import_scan`:
    - Detectar automáticamente si la carpeta seleccionada contiene un escaneo dual (`dual_scan_*` con `nube_fusionada.ply`, `nube_cam1.ply` y `nube_cam2.ply`).
    - Ofrecer opción de importar:
    - **Nube Fusionada Completa** (300k+ puntos combinados).
    - **Nubes Separadas por Colección** (`Col_Cam1_Master` y `Col_Cam2_Slave`) para inspeccionar la superposición en Blender.

3. Añadir selector de **Modo de Detalle / LOD**:
    - **LOD 1 (Rápido Delaunay 2.5D):** Malla rápida con proyección UV.
    - **LOD 2 (Poisson Alta Definición):** Malla estanca con octree depth 9 y limpieza topológica.
    - **LOD 3 (Geometría con Mapa de Relieve Infrarrojo):** Incorpora la imagen infrarroja como mapa de relieve o rugosidad en el material Principled BSDF.

---

4. Checklist de Verificación y Criterios de Aceptación

Para dar por concluida la implementación de la captura y renderizado dual con Claude, se deben cumplir las siguientes pruebas:

- [ ] **Test 1: Streaming Concurrente Dual a 30 FPS en WebGL:**
- Ambas cámaras visibles al mismo tiempo en el navegador en http://localhost:5002.
- Tasa de refresco medida superior a 50 FPS en el canvas WebGL.
- [ ] **Test 2: Coherencia de Coordenadas Espaciales:**
- Al colocar un objeto entre las dos cámaras, las superficies capturadas por la Cámara 1 y la Cámara 2 deben coincidir geométricamente en el mismo volumen tridimensional.
- El modo de color "Diagnóstico de Solape" muestra el cruce nítido entre la capa cian y magenta.
- [ ] **Test 3: Ajuste Fino en Caliente:**
- Mover un slider de traslación desplaza suavemente la nube de la Cámara 2 en tiempo real.

- Al pulsar "Guardar Alineación", el archivo `config/dual_camera_config.json` se actualiza con la nueva matriz.
- [ ] **Test 4: Captura Dual Estática a Disco:**
- El botón "Captura Dual 360°" genera en `Saves/<fecha>/dual_scan_<hora>/` los archivos `nube_fusionada.ply` (>300.000 puntos), `nube_cam1.ply`, `nube_cam2.ply`, imágenes y metadatos.
- [ ] **Test 5: Importación en Blender:**
- El addon `RealSense_Importer.py` carga el escaneo dual sin errores de Python en Blender 3.x / 4.x.